

Validating SMT Rewriters via Rewrite Space Exploration Supported by Generative Equality Saturation

MAOLIN SUN, Nanjing University, China
YIBIAO YANG, Nanjing University, China
JIANGCHANG WU, Nanjing University, China
YUMING ZHOU, Nanjing University, China

Satisfiability Modulo Theories (SMT) solvers are widely used for program analysis and other applications that require automated reasoning. Rewrite systems, as crucial integral components of SMT solvers, are responsible for simplifying and transforming formulas to optimize the solving process. The effectiveness of an SMT solver heavily depends on the robustness of its rewrite system, making its validation crucial. Despite ongoing advancements in SMT solver testing, rewrite system validation remains largely unexplored. Our empirical analysis reveals that developers invest significant effort in ensuring the correctness and reliability of rewrite systems. However, existing testing techniques do not adequately address this aspect. In this paper, we introduce Aries, a novel technique designed to validate SMT solver rewrite systems. First, Aries employs ***mimetic mutation***, a targeted strategy that actively reshapes input formulas to provoke and diversify rewrite opportunities. By aligning mutated terms with known rewrite patterns, Aries can conduct a thorough exploration of the rewrite space in the following phase. Second, Aries utilizes ***deductive rewriting***, leveraging *generative* equality saturation to effectively explore rewrite space and produce semantically equivalent mutants for the purpose of validation. We implemented Aries as a practical validation tool and evaluated it on leading SMT solvers, including Z3 and cvc5. Our experiments demonstrate that Aries effectively identifies bugs, with 27 new issues detected, of which 22 have been confirmed or fixed by developers. Most of these issues involve the rewrite systems, highlighting Aries's strength in exploring the rewrite space.

CCS Concepts: • **Software and its engineering → Formal methods**; **Software testing and debugging**.

Additional Key Words and Phrases: SMT solver, validation, rewrite system, equality saturation

ACM Reference Format:
Maolin Sun, Yibiao Yang, Jiangchang Wu, and Yuming Zhou. 2025. Validating SMT Rewriters via Rewrite Space Exploration Supported by Generative Equality Saturation. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 315 (October 2025), 27 pages. https://doi.org/10.1145/3763093

1 Introduction

Satisfiability Modulo Theories (SMT) solvers are pivotal theorem proving systems that automate the process of determining the satisfiability of logical formulas within various theories, such as arithmetic, arrays, bit-vectors, and strings [Barrett and Tinelli 2018]. These solvers find extensive applications across programming language practices, such as program analysis [Wang et al. 2023; Yao et al. 2024], symbolic execution [Cadar et al. 2008], and system verification [Nelson et al. 2017;

Authors' Contact Information: Maolin Sun, State Key Laboratory for Novel Software Technology, School of Computer Science, Nanjing University, Nanjing, China, merlin@smail.nju.edu.cn; Yibiao Yang, State Key Laboratory for Novel Software Technology, School of Computer Science, Nanjing University, Nanjing, China, yangyibiao@nju.edu.cn; Jiangchang Wu, State Key Laboratory for Novel Software Technology, School of Computer Science, Nanjing University, Nanjing, China, jiangchangwu@smail.nju.edu.cn; Yuming Zhou, State Key Laboratory for Novel Software Technology, School of Computer Science, Nanjing University, Nanjing, China, zhouyuming@nju.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2025 Copyright held by the owner/author(s).
ACM 2475-1421/2025/10-ART315
https://doi.org/10.1145/3763093

Sigurbjarnarson et al. 2016; Sun et al. 2024]. The efficacy of SMT solvers significantly hinges on their rewrite systems (*a.k.a.* rewriters), which optimize expressions by methodically applying a predefined set of rewrite rules. Each rule, typically denoted as $l \rightsquigarrow r$, substitutes occurrences of the syntactic pattern l with instances of a semantically equivalent pattern r, e.g., $(x * 0) \rightsquigarrow 0$. Solvers' developers assert that optimizing rewriters is as essential as refining the core decision procedures [Nötzli et al. 2019], underscoring their value in enhancing solver performance and managing complex theories. For instance, without effective rewriting, the performance of the leading solver cvc5 [Barbosa et al. 2022], deteriorates dramatically, experiencing a slowdown exceeding 100-fold in some cases [Nötzli et al. 2019].

However, developing correct and efficient rewrite systems is challenging, as stated by developers [Nötzli et al. 2019]. Rewriters must balance theoretical correctness with performance optimization, requiring deep expertise in both SMT solvers and the underlying theories. To achieve better performance, developers should continuously refine and expand the rewrite systems, such as introducing new rules, eliminating redundancies, and optimizing heuristics. Our empirical analysis reveals that approximately 40% of the Pull Requests (PRs) in cvc5 pertain to modifications of rewriters, suggesting a significant allocation of developers' efforts, as further detailed in the subsequent section. Given this substantial investment, errors are also inevitable, leading to potential bugs or overlooked functionalities that can affect solver reliability and performance [Lachnitt et al. 2024]. Thus, rigorous validation of SMT solver rewriters is crucial to ensuring solver quality.

Given the critical application of SMT solvers in various research domains, numerous methodologies have been developed to test the solvers [Brummayer and Biere 2009; Winterer and Su 2024]. While these methodologies have been effective in detecting bugs in solvers, they are not designed for validating rewrite systems. To this end, they do not guarantee the inclusion of specific terms within the generated test formulas to activate particular rewrite rules, limiting the effectiveness of triggering the rewrite mechanisms. Consequently, these rewrite systems remain insufficiently exercised. This study seeks to address this gap by focusing specifically on the validation of rewrite systems within SMT solvers. However, this endeavor is extremely tricky. In particular, validating rewrite systems demands the creation of a varied set of test formulas to effectively explore the rewriters' capabilities, which poses a significant challenge.

Our Approach. In this study, we propose Aries[1], a novel method for validating rewrite systems in SMT solvers. Our approach begins with a foundational set of rewrite rules. Using these rewrite rules, we leverage Aries to explore rewrite space, which consists of ***mimetic mutation*** and ***deductive rewriting***, to effectively validate rewriters. Mimetic mutation modifies existing formulas, such as those from open-source solver test suites, by reshaping terms to align with specific rewrite rule patterns. The intuition behind this design is to increase the likelihood of exercising rewrite rules with non-trivial patterns, ensuring a more comprehensive validation. Moreover, this heuristic strategy also enables rule application on formulas that, in their original form, would not have triggered any rewrites, thereby creating more opportunities for the subsequent generation phase.

After mimetic mutation, Aries applies deductive rewriting using *generative* equality saturation to a given formula, generating novel test formulas. Particularly, conventional equality saturation [Willsey et al. 2021] is typically used to find the simplest equivalent form of an expression for purposes such as compiler optimizations. In this context, we adapt it to more effectively meet the requirements for generating structurally distinct yet semantically equivalent formulas. In other words, rather than only simplifying, our goal is to support expanding simple expressions into more complex ones and to extract a variety of equivalent forms. To enable this, Aries constructs equality graphs (*e-graphs*) [Nelson and Oppen 1980] to represent terms within the formulas and

[1] Aries := V**A**lidate **R**ewriter via Rewr**I**t**E** **S**pace Exploration

iteratively populates the e-graphs using a collection of preprocessed rewrite rules. Importantly, our adaptation supports unconventional rewrites such as $0 \rightsquigarrow (x * 0)$ and $False \rightsquigarrow (x \vee \neg x)$, which would typically be avoided in optimization scenarios. We encode such patterns using symbolic placeholders, e.g., $Zero(x) \rightsquigarrow (x * 0)$, to make them tractable for equality saturation engines. Once the e-graph is saturated, Aries samples diverse, semantically equivalent terms from it and uses them to construct new test formulas. These rewritten formulas are then submitted to SMT solvers alongside their original counterparts. As the original and rewritten formulas are logically equivalent, they should produce consistent results. Thus, any inconsistency in solver output may reveal bugs in the rewrite systems. This provides a strong oracle for identifying potential bugs, whether they affect correctness or performance.

Additionally, to accurately determine the root causes of these discrepancies, we have devised a reduction heuristic aimed at isolating suspicious rewrites implicated by deductive rewriting, while concurrently minimizing irrelevant portions of the formulas. This practice is critical for assisting developers in solidifying and improving rewrite systems [Zhang et al. 2023b]. Concretely, for the modified sections of the formulas, our strategy iteratively searches for simplified terms within e-graphs, enabling the identification of suspicious rewrites and excluding innocent ones. For the unaltered sections, the reduction strategy leverages existing test input reduction techniques to further minimize the formulas, while preserving sections relevant to the suspect rewrites. In this manner, the reduction heuristics effectively minimize the test formulas and pinpoint the truly suspicious rewrites. Ultimately, we report these bugs to the developers, along with recommendations for potential fixes and enhancements.

We have developed Aries, a practical validation tool designed for SMT solvers, that also features the capability to effectively minimize test inputs. Our evaluation of Aries's effectiveness involved its application to the leading SMT solvers, i.e., Z3 [de Moura and Bjørner 2008] and cvc5 [Barbosa et al. 2022]. Specifically, Aries has successfully detected 27 genuine bugs in the solvers. Of these, 22 were subsequently confirmed by the developers, with 16 of the confirmed bugs being fixed, most of which were rewrite-related issues.

Contributions. Our main contributions are as follows:

- ***Novelty:*** We highlight the critical role of rewrite systems in SMT solvers and present a new validation framework that includes *mimetic mutation* and *deductive rewriting* to effectively explore the solver's rewrite space. To support this, we collect a comprehensive set of rewrite rules and adapt equality saturation to synthesize effective test formulas that extensively exercise rewriting behaviors in solvers.
- ***Significance:*** Our work demonstrates how equality saturation can be effectively applied to testing and delta debugging. This broadens its applicability and shows its potential for uncovering and analyzing correctness and performance issues in SMT solvers.
- ***Practicality:*** We have implemented Aries as a practical tool and used it to uncover 27 issues in the leading SMT solvers, namely Z3 and cvc5. Of these, 22 issues have been confirmed or resolved by the developers. Based on the examination of developers' resolution, we observe that the majority of the bugs are primarily associated with the rewrite systems.

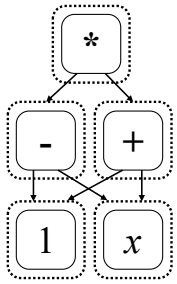
Paper Organization. The rest of this paper is structured as follows. Section 2 introduces the background and our motivation. Section 3 formalizes our approach and illustrates our approach using a running example. Section 4 provides details on the implementation of Aries. Next, we elaborate on our evaluation in detail in Section 5, and Section 6 further discusses the results. Finally, we survey related work in Section 7, and the conclusion is in Section 8.

```
(declare-fun x () Int)
(assert (> x 0))
(assert (= 1 (* (- 1 x) (+ 1 x))))
(check-sat)
```

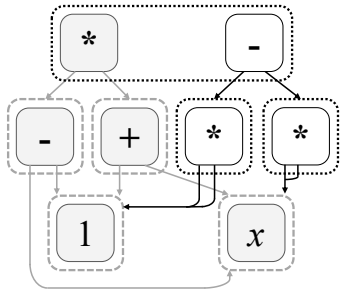
(a) An SMT-LIB formula.

```
(/ x 1) -> x
(* (- 1 x) (+ 1 x)) -> (- (* 1 1) (* x x))
...
```

(b) Rewrite rules applied.



(c) Initial e-graph.



(d) Updated e-graph.

Fig. 1. Example of partial processes within equality saturation. Additions and modifications in the e-graphs are highlighted in black.

2 Background and Motivation

In this section, we first briefly introduce the SMT-LIB language and the concept of equality saturation. Then, we motivate this study using a real-world example and an empirical investigation.

2.1 Background

SMT-LIB Language. The SMT-LIB standard [Barrett et al. 2022] is a widely adopted initiative in the automated reasoning community that provides a common framework for describing problems in Satisfiability Modulo Theories (SMT). By offering a unified language, SMT-LIB enables consistent assessment of SMT solvers across diverse benchmarks and logical theories. Over time, it has become the primary input language for SMT solvers, with a rich set of commands and constructs for expressing complex logical formulas. Among the most fundamental commands in the SMT-LIB language are **`declare-fun`**, **`assert`**, and **`check-sat`**. Figure 1a illustrates these commands with a simple example. At the beginning, **`(declare-fun x () Int)`** declares an integer variable r. The **`assert`** commands introduce constraints that the variables or functions must meet. In this case, the first **`assert`** command enforces the condition that the integer variable x must satisfy $x > 0$. Formulas generally comprise several **`assert`** commands, all of which need to be met simultaneously. The **`check-sat`** command then prompts the solver to determine the formula's satisfiability. An SMT solver will indicate **`sat`** if it is possible to satisfy all constraints simultaneously. Conversely, if no assignment can fulfill all assertions, the instance is deemed **`unsat`**.

Equality Saturation. Equality saturation [Tate et al. 2009] is a powerful technique that leverages equality graphs (*e-graphs*) to implement efficient term-rewriting engines. Starting from an initial term t and a predefined set of rewrite rules, an equality saturation engine constructs an initial e-graph E that represents t. It then iteratively applies the rewrite rules to expand E into a larger set of equivalent terms. To illustrate, consider the highlighted term $((1 - x) * (1 + x))$ in Figure 1a. An e-graph can be constructed to represent this term, as demonstrated in Figure 1c. Next, the e-graph may be transformed into the form shown in Figure 1d through application of the rewrite rule highlighted in Figure 1b. Each node enclosed in a solid box, along with its associated subtree, signifies a term. Terms grouped within the same dashed box form an equivalence class (*e-class*), indicating that all terms represented by nodes in an e-class are equivalent. For instance, in Figure 1d, the operators "$*$" and "$-$" are in the same e-class, establishing the equivalence between the terms $((1 - x) * (1 + x))$ and $((1 * 1) - (x * x))$. By maintaining the e-graph structure, the equality saturation engine efficiently stores all equivalent terms associated with a given term, thereby

```
; alpha_1
(declare-fun a () String)
(assert (= a (str.replace_all "" a "")))
(check-sat)
```

(a) Formula α_1 classified as unknown by Z3.

```
; alpha_2
(declare-fun a () String)
(assert (= a ""))
(check-sat)
```

(b) The semantically equivalent formula α_2.

```
$ z3 alpha1.smt2
unknown

$ z3 alpha2.smt2
sat
# get model
(
a ""
)
```

(c) Command lines.

```
br_status seq_rewriter::mk_seq_replace_all
{
  ...
  if (str().is_empty(a) && str().is_empty(c))
  {
    result = a;
    return BR_DONE;
  }
  ...
}
```

(d) Patched rewrite rule in Z3.

Fig. 2. A motivating example. #7391

avoiding redundant computations. Note that the e-graph depicted in Figure 1d has not yet reached the state of equality saturation. In practice, the rewriting process terminates either upon achieving saturation, where no new nodes or edges can be added to the e-graph, or upon reaching a predefined timeout. Once the process completes, the engine can extract specific terms based on a user-provided cost function tailored to user requirements. For example, in optimization contexts, the engine might extract the simplest term that retains the semantics of the original term. In recent years, equality saturation has gained significant traction in the research community, with numerous studies investigating its potential across various domains [Abeysinghe et al. 2024; Cao et al. 2023; Chen et al. 2024; Cheng et al. 2024; VanHattum et al. 2021].

2.2 Motivation

Rewrite Systems in SMT Solvers. Rewrite systems are integral for converting terms or expressions into their semantically equivalent counterparts, finding widespread application in compilers [Chen et al. 2018], program synthesizers [Hoe and Arvind 1999], and databases [Wang et al. 2022]. Within the context of SMT solvers, rewrite systems play a crucial role in simplifying input formulas through rewrite rules to enhance the efficiency of problem-solving [Barbosa et al. 2022; Nadel 2014]. Typically, solvers can enable rewriting during both pre-processing and solving [cvc 2024]. Effective rewrite systems combine theory-specific simplifications with heuristic optimizations, integrating closely with decision-making, model construction, and proof generation [Barbosa et al. 2022; Nötzli et al. 2022]. Designing such systems requires balancing targeted simplifications with broader heuristic improvements.

Without appropriate rewrite systems, SMT solvers may fail on simple formulas. Figure 2 presents a case where an incomplete rewrite rule affects solver behavior. Concretely, for the straightforward formula α_1 depicted in Figure 2a, the Z3 solver returns unknown as it cannot determine whether it is satisfiable. We can observe that (`str.replace_all` “” x “”) can simplify to an empty string: replacing all occurrences of any substring x in an empty string with another empty string always

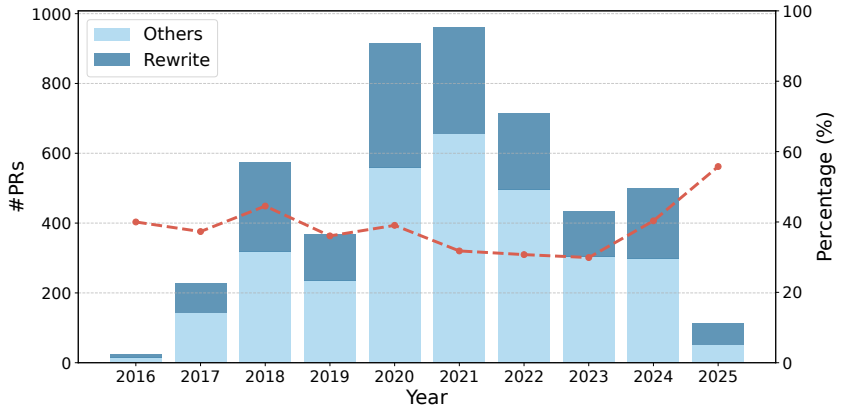


Fig. 3. A ten-year distribution of Pull Requests in cvc5 is depicted, with the dashed line illustrating the proportion of PRs pertaining to rewrite systems relative to the overall PRs.

yields an empty string. Thus, the constraint should essentially be `(assert (= a ""))`, which is satisfiable. Correspondingly, by manually applying the rewrite `(str.replace_all "" x "")` $\rightsquigarrow$ `""`, as shown in Figure 2b and 2c, the solver correctly returns `sat` for the simplified formula α_2. Thus, we can deduce that this completeness issue can be addressed by adding the aforementioned rewrite. Note that this issue has been unearthed by our technique Aries and reported to the developers. We recommended adding the missing rewrite rule, which was subsequently adopted by the developers, as depicted in Figure 2d. This real-world example highlights the critical role of rewrite systems in SMT solvers, directly impacting their performance and even correctness. However, as noted by developers, implementing these systems is complex and error-prone [Nötzli et al. 2019]. This challenge motivates our work on an effective approach to validating rewrite systems.

Preliminary Study on Rewrite Systems. To better understand the importance of robustifying rewrite systems in SMT solvers, we conduct an empirical study on the maintenance of rewrite rules in the cvc5 solver. We select cvc5 as it is a state-of-the-art SMT solver that has been well-maintained with clear Pull Requests (PRs). We collect the PRs from the cvc5 repository that range through the last ten years (i.e., from 2015 to 2025) and focus on the PRs that involve feature enhancements or bug fixes instead of those related to documentation or code refactoring. To achieve this, we adopt the methodology leveraged in prior work [Batoun et al. 2023] to categorize PRs. To illustrate, we commence by selecting the PRs that feature keywords such as "fix" and "enhancement" in their titles or descriptions. By contrast, we exclude PRs with keywords such as "doc" and "refactor", signifying that these PRs do not introduce essential changes to the solver. Note that we primarily focus on the examination of merged PRs to guarantee the accuracy and reliability of the findings. Ultimately, we identified 4,841 PRs pertinent to bug fixes and feature alterations out of an approximate total of 8,000 PRs. We then extended our analysis to evaluate the relevance of these PRs concerning rewrite systems by examining their titles, descriptions, and code modifications. For example, we deem a PR relevant to rewrite systems if it includes code additions, modifications, or removals in files whose names contain the keyword "rewrite".

Figure 3 illustrates the distribution of PRs in cvc5, including those related to rewrite systems, over the last ten years. Notably, the figure omits the data for the year 2025 since the total number of PRs is merely two for that year. We observe that the proportion of PRs related to the rewrite system has remained fairly constant in most years, constituting about 40% of the total PRs. Especially, in 2025,

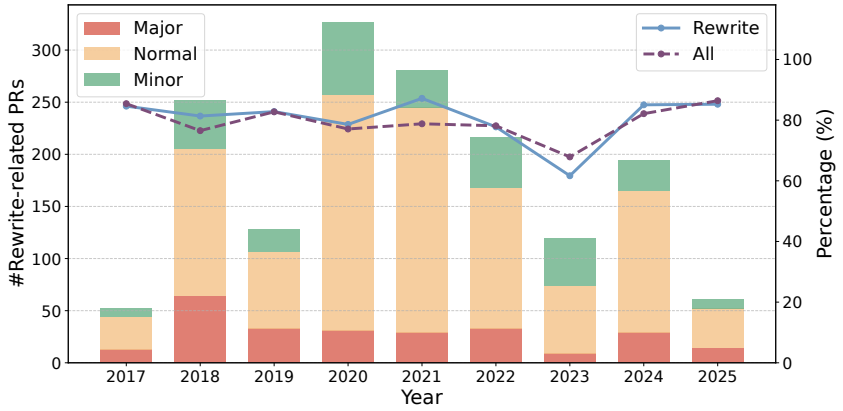


Fig. 4. Proportional distribution of priorities within rewrite-related Pull Requests in cvc5. The dashed line represents the ratio of PRs possessing major and normal priorities to the entire pool of PRs. The solid line reflects the ratio of rewrite-related PRs with major and normal priorities relative to the total number of rewrite-related PRs.

the percentage of rewrite system-related PRs approaches 60%. However, this trend primarily arises from the limited sample size, which introduces a degree of distortion. This observation further underscores the importance of maintaining rewrite systems in SMT solvers, as evidenced by the consistent efforts devoted to enhancing and fixing rewrite systems in cvc5. Figure 4 presents the priority distribution of rewrite-related bugs in cvc5, where priority labels, assigned by developers, suggest the importance of the modifications. The priority levels are classified into three categories: "major," "normal," and "minor." Additionally, the figure also includes lines representing the percentage of PRs with major and normal priorities, as well as the percentage of rewrite-related PRs with these priorities. Notably, since PRs before 2017 were not assigned priority labels, they are excluded from the analysis. According to the statistics, we observe that the percentage of rewrite-related PRs with major and normal priorities is similar to that of the total PRs, indicating that the maintenance of rewrite systems is as crucial as other maintenance tasks in cvc5. Furthermore, this observation aligns with developers' views that optimizing rewriters is as essential as optimizing core decision procedures in SMT solvers [Nötzli et al. 2019]. These findings underscore the crucial role of rewrite systems in SMT solvers, further motivating our study.

3 Approach

In this section, we first provide a comprehensive overview of our proposed approach, Aries. We then elaborate on how Aries performs mimetic mutation and deductive rewriting based on the collected rewrite rules to explore the solver's rewrite space. Furthermore, we use a running example to illustrate the process of Aries. Finally, we introduce a tailored strategy within Aries for reducing the size of bug-revealing formulas.

3.1 Overview

This study introduces Aries, an innovative methodology designed for the validation of SMT rewrite systems. The validation workflow of Aries is depicted in Figure 5. Specifically, Aries starts with existing SMT-LIB instances and a set of preliminarily collected rewrite rules conforming to the SMT-LIB standard. Subsequent to the accumulation of rewrite rules, Aries proceeds to mutate and generate novel test formulas through a two-phased process: *mimetic mutation* and *deductive*

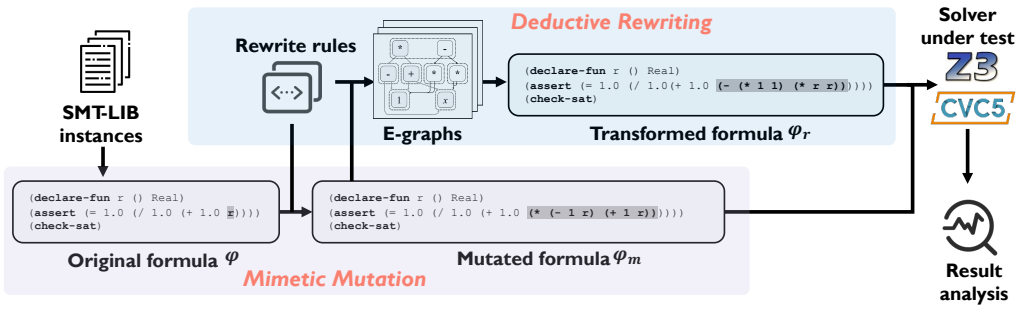


Fig. 5. Overview of Aries's testing process.

rewriting. Algorithm 1 presents the process of test formula generation. Concretely, given each formula φ in the existing seed set, the *mimetic mutation* phase aims to alter certain terms in the original formula φ to align with the specific rewrite rules' patterns (Line 4). Thus, it is supposed to provide additional rewrite opportunities and expand the scope for Aries's transformations in the following phase. During the *deductive rewriting* phase (Line 6), Aries builds e-graphs for the tailored encoding of terms in formula φ_m and employs generative equality saturation, which facilitates the generation and sampling of diverse formulas with equivalent semantics. Finally, Aries engages the SMT solver to process the generated formulas φ_m and φ_r, monitoring for any aberrant behavior (Lines 7-9). If any such behaviors emerge, Aries proceeds by isolating the rewrite rules accountable for the identified anomalies. In particular, it eliminates irrelevant rewriting operations and simplifies the formula segments that remain unaltered, respectively. Ultimately, these reduced test formulas, along with the potentially problematic rewrite rules, are reported to the solvers' developers for further examination. The following subsections describe each phase of the workflow in detail.

3.2 Rewrite Rule Collection

Rule Source. To enable exploration of the rewrite space within SMT solvers, it is imperative to commence with a collection of rewrite rules. Particularly, for effective validation, it is essential to ensure that these rules are both representative and abundant. To meet these requirements, we initially incorporate rewrite rules collected by prior work [Lachnitt et al. 2024], which encompasses 248 rewrite rules. As the literature indicates, these rules are primarily derived from the SMT solver cvc5 [Lachnitt et al. 2024]. Nonetheless, the rewrite rules employed in most SMT solvers lack explicit documentation [Brain 2024], posing challenges to the assembly of a plentiful set of rewrite rules. To overcome this limitation, Aries employs automated rule inference tools, such as Ruler [Nandi et al. 2021], to supplement the generation of rewrite rules. These tools can be configured to synthesize rewrite rules applicable to the domain of SMT problems across various theories, thereby expanding the scope of possible validation scenarios. This practice enables the collection of additional logically sound rewrite rules, some of which may not yet be implemented in existing solvers, thus contributing to uncover potential improvements in the rewrite systems.

Rule Verification. Since the rules provided by prior work [Lachnitt et al. 2024] are mainly sourced from cvc5, it is not guaranteed that they are free from solver-specific behaviors and completely adhere to the SMT-LIB standard. To ensure rules' compatibility across different solvers and minimize false positives, we employ another SMT solver Z3 for cross-verification. Specifically, given a rewrite rule $A \rightsquigarrow A'$, Aries formulates this rule as the expression `(not (= A A'))`, i.e., $\neg(A = A')$, which is then fed to an SMT solver for satisfiability checking. If the solver returns `unsat`, it indicates

Algorithm 1: ARIES's pseudocode

```
1  Procedure Aries(Seeds, Rules, Solver)
2      Bugs ← ∅
3      foreach φ in Seeds do
4          φ_m ← MimeticMutate (φ, Rules)
5          Res_O, Stats_O ← Validate(φ_m, Solver)
6          φ_r ← DeductiveRewrite (φ_m, Rules)
7          Res_r, Stats_r ← Validate(φ_r, Solver)
8          if Res_r ≠ Res_O then
9              Bugs ← Bugs ∪{φ_r}
```

Algorithm 2: The pseudocode of *mimetic mutation*

```
1  Procedure MimeticMutate(φ, Rules)
2      φ_m ← φ
3      for i to iteration_m do
4          term ← RandomSelect (φ_m)
5          rule ← RandomSelect (Rules)
6          target ← rule.left
7          for var in target.vars do
8              if isEmpty (term.vars) then
9                  replacer ← RandomVar (var.type)
10             else
11                 replacer ← RandomSelect (term.vars)
12                 if replacer.type ≠ var.type then
13                     replacer ← Convert (replacer, var.type)
14             target ← target.replace(var, replacer)
15         if target.type ≠ term.type then
16             target ← Convert (target, term.type)
17         φ_m ← φ_m.replace(term, target)
18     return φ_m
```

that there is no assignment that violates the rule, confirming that `A` is equivalent to `A'` under all conditions. Thus, the consistent **`unsat`** results from both solvers corroborate the rule's validity, permitting its inclusion in our rule set. Conversely, if any solver returns **`sat`**, the rule is deemed improper and is either revised manually or discarded. For conditional rewrite rules, which only hold under specific conditions, we incorporate the associated conditions into the formulated constraints. Thus, that leads to the constraint **`(and (not (= A A')) C)`**, where **`C`** represents the rule's condition.

This process ultimately produces a verified set of rewrite rules, which ARIES utilizes in later stages of formula generation. It is important to properly handle unconventional rules that introduce fresh variables, such as $0 \rightsquigarrow (x * 0)$. For instance, to manage this, we represent the left-hand side term 0 as a parameterized form `Zero(x)`, resulting in the rule $Zero(x) \rightsquigarrow (x * 0)$. This encoding enables controlled variable instantiation and facilitates further manipulation in subsequent steps.

3.3 Mimetic Mutation

In practice, many existing formulas are supposed to lack specific patterns that align with certain gathered rewrite rules. This issue is particularly pronounced for rewrite rules with complex or non-trivial patterns, which restricts the exercise of these rewrite rules during validation. To address this issue, Aries performs mimetic mutation to generate new formulas that possess more potential for rewriting, boosting the subsequent deductive rewriting process.

Definition 3.1 (Mimetic mutation). *Let there be an SMT formula φ and a rewrite rule $l \rightsquigarrow r$. We designate formula $\varphi' = \varphi[t_l/t]$ as a* **mimetic mutant** *of φ if t_l aligns with the pattern of the rule's left-hand side l, where t is a term within φ. The transformation of φ into $\varphi[t_l/t]$ by substituting t with t_l is defined as* **mimetic mutation**.

The procedure of mimetic mutation is delineated in Algorithm 2. The process begins with Aries generating a duplicate of the original formula φ, referred to as φ_m (Line 2). Following this, Aries embarks on an iterative procedure. In each iteration, Aries selects a term within φ_m and a rewrite rule from the set of available rules at random (Lines 4-5). The goal of mimetic mutation is to modify the selected *term* to match the pattern presented on the left-hand side of the specified rewrite rule, denoted as *target*. This modification involves replacing the variables in *target* with those in *term* if *term* contains any variables (Line 11). Otherwise, Aries randomly produces a variable with the same type as the target variable (Lines 8-9). Similarly, it functions in the same manner for targets that include encoded terms, e.g., `Zero(x)`. It is important to note that the types of the selected substituted variables should be consistent with the types of the target variables to ensure a well-typed substitution. Upon discovering a discrepancy in variable types, Aries performs type conversion using specific operators defined in the SMT-LIB standard (e.g., `to_int`, `to_real`, `is_int`) to ensure substitution correctness (Lines 12-13). Table 1 presents the conversion functions utilized by Aries. The selection of an appropriate conversion function is based on the types of both the target and selected variables, with the function being applied to the selected variable to achieve type consistency. In cases where several conversion functions are applicable, Aries randomly selects one. Subsequently, *term* is replaced by *target* in φ_m, resulting in a new mimetic mutant. Notably, the type of *target* should also be converted if it is not consistent with the type of *term* (Line 15-16). If type conversion is impossible to achieve a well-typed mutant using the provided conversion functions, it will be discarded.

Example. To illustrate our procedure more clearly, we present a running example. We begin by mutating terms in the original formula φ to match patterns randomly selected from our collection of rewrite rules, as shown in Figure 6a. For instance, consider selecting the last rewrite rule depicted in Figure 6b, i.e., $(1 - x) * (1 + x) \rightsquigarrow (1 * 1) - (x * x)$. We can select a term in φ and reshape it to match the left-hand side of the rule by substituting variables accordingly. Suppose we select the term `r` from φ and reshape it to the pattern $(1 - x) * (1 + x)$. This ultimately yields the mutated formula φ_m, as shown in Figure 6c. Note that this kind of mutation can be applied multiple times to maximize rewrite opportunities. Besides, maintaining type consistency throughout this mutation process is essential to preserving the validity of the mutated formula. In this example, no additional adjustments to the type of `r` are needed, as the rule's variable can be either real or integer.

3.4 Deductive Rewriting

At the heart of our validation framework lies deductive rewriting, which enables Aries to explore the SMT solver rewrite space using *generative* equality saturation. The equality saturation technique efficiently generates equivalent formula mutants with varying complexity. It employs an e-graph

Table 1. Conversion function symbols as defined in the SMT-LIB standard.

Function Types	Function Symbols
$\Gamma, Int <: Real \vdash Real \rightarrow Bool$	`is_int`
$\Gamma \vdash Str \rightarrow Bool$	`str.is_digit`
$\Gamma \vdash Real \rightarrow Int$	`to_int`
$\Gamma \vdash Str \rightarrow Int$	`str.to_int, str.to_code`
$\Gamma \vdash BitVec \rightarrow Int$	`bv2nat`
$\Gamma \vdash Int \rightarrow Real$	`to_real`
$\Gamma \vdash Int \rightarrow Str$	`str.from_int, str.from_code`
$\Gamma \vdash Int \rightarrow BitVec$	`int2bv`

```
(declare-fun r () Real)
(assert (= 1.0 (/ 1.0
          (+ 1.0 r))))
(check-sat)
```

(a) Original formula φ.

```
(/ x 1) -> x
Zero(x) -> (* x 0)
(* (- 1 x) (+ 1 x)) -> (- (* 1 1) (* x x))
...
```

(b) Rewrite rules preliminarily prepared.

```
(declare-fun r () Real)
(assert (= 1.0 (/ 1.0 (+ 1.0
    (* (- 1 r) (+ 1 r)) ))))
(check-sat)
```

(c) Formula φ_m by mimetic mutation.

```
(declare-fun r () Real)
(assert (= 1.0 (/ 1.0 (+ 1.0
         (- (* 1 1) (* r r)) ))))
(check-sat)
```

(d) Formula φ_r by deductive rewriting.

Fig. 6. Rewrite Space Exploration illustrated.

data structure that represents terms and their equivalences. In our context, each e-graph node[2] represents a *term* in an SMT formula, and nodes in the same equivalence class denote equivalent terms. Upon reaching saturation (i.e., the state where no additional rewrites can modify the e-graph), the e-graph encompasses all expressions that are equivalent to the initial one.

The procedure is delineated in Algorithm 3, commencing with the original formula φ_m and a set of preprocessed rewrite rules, *Rules*. Specifically, Aries constructs e-graphs based on the provided formula φ_m (Line 5). To manage the overall performance of Aries and facilitate subsequent bug analysis, we apply a selective strategy, constructing e-graphs only for specific terms within φ_m rather than for the entire formula. In particular, we prioritize terms that include subterms modified during mimetic mutation for e-graph construction. Subsequently, the equality saturation engine iteratively applies the collected rewrite rules to the e-graphs, continuously updating nodes and edges until a saturation state is achieved (Line 6).

Upon reaching saturation, Aries selects equivalent terms from the e-graphs to substitute for the original terms. Here, to promote the diversity of mutants, we incorporate a filtering mechanism that intentionally excludes similar mutations. Concretely, we employ the proportion of distinct s-expressions (i.e., symbolic expressions) within the extracted term *candidate* compared to the

[2]An e-graph node is a fundamental element of an e-graph, typically comprising operators and their associated operands.

Algorithm 3: The pseudocode of *deductive rewrite*

```
1  Procedure DeductiveRewrite(φ_m, Rules)
2      φ_temp ← φ_m
3      terms ← RandomSelect (φ_m)
4      foreach term in terms do
           // Build E-graph for each term and apply equality saturation
5          EGraph ← BuildEGraph (term)
6          EGraph.saturate(Rules)
7          Score_max ← 0
8          while no termination criterion met do
9              candidate ← RandomExtract (EGraph)
10             Score ← DiverEstimate (candidate.Sexpr, term.Sexpr)
11             if Score > Score_max then
12                 Score_max ← Score
13                 replacer ← candidate
14             if random.uniform(0, 1) ≤ Score then
15                 replacer ← candidate
16                 break
17         φ_temp ← φ_temp.replace(term, replacer)
18     φ_r ← φ_temp
19     return φ_r
```

original term to represent the diversity score of terms (Lines 9-10). S-expressions, basic elements documented in the SMT-LIB standard [Barrett et al. 2022], are used to represent nested list data structures. For example, the term `(* (- 1 r) (+ 1 r))` described in Figure 6c contains seven distinct s-expressions: `*`, `-`, `1`, `r`, `+`, `(- 1 r)`, and `(+ 1 r)`. In this context, a larger mutation-induced difference in s-expressions increases the probability that the filtering mechanism will accept the generated mutants (Lines 14-16). For instance, if the selected candidate is `(- (* 1 1) (* r r))`, which includes six distinct s-expressions, i.e., `-`, `*`, `1`, and `r`, `(* 1 1)`, and `(* r r)`, the diversity score is 0.33. This indicates that the term will be accepted with a probability of 33%. If not accepted, the term will be discarded, and the process will repeat until a relatively diverse term is found or the maximum number of attempts is reached. In the latter case, the term that exhibits the highest diversity score among previous attempts will be chosen (Lines 11-13). Eventually, the formula is transformed into a new formula φ_r by substituting the original terms with the selected equivalent terms (Line 18), thereby maintaining satisfiability. Concurrently, if any encoded expressions are present in the new formula, these must be translated back to conform to the SMT-LIB standard.

Example. In this phase, we first construct an e-graph to represent the terms in the mutated formula φ_m. Suppose that we build an e-graph for the term introduced by the mutation, i.e., $((1-r)*(1+r))$, as illustrated in Figure 6c. Once the e-graph is initialized, the equality saturation engine applies the prepared rewrite rules to expand the set of equivalent terms within the e-graph. Concretely, the engine attempts to match both the term and its subterms with the left-hand side patterns of the rewrite rules, subsequently generating new terms by applying the rules. For instance, applying the last rewrite rule in Figure 6b to the e-graph can produce an updated e-graph, similar to Figure 1d. In particular, the newly added e-class represents the term $((1*1)-(r*r))$, which is semantically equivalent to the original term $((1-r)*(1+r))$. This process continues until the engine reaches a

state of equality saturation, which may consist of a large number of equivalent formulas. At that point, we can extract diverse equivalent terms from the e-graph to substitute the original term in the formula φ_m. In this example, assume that we successfully sample the term $((1 * 1) - (r * r))$ from the fully populated e-graph to replace the term $((1 - r) * (1 + r))$ in the formula φ_m. This will yield the transformed formula φ_r shown in Figure 6d. Notably, while this example only involves a single rewrite rule for simplicity, in practice, multiple rules will apply in combination. For instance, the saturated e-graph might also contain the term $((1 * 1) - ((-r) * (-r)))$, which is equivalent to $((1 * 1) - (r * r))$ but syntactically more complex. Due to its higher diversity score, such a term is more likely to be sampled in this context.

By following the aforementioned steps, we derive the formula φ_r, which maintains semantic equivalence with the formula φ_m. Given their equivalence, a solver's behavior on φ_r can then be compared to its behavior on φ_m to detect potential bugs or inefficiencies. In practice, the transformed formula, φ_r, exposes a performance issue in the cvc5 solver. Specifically, the solver exhibits an unexpected behavior when processing φ_r, leading to a prolonged execution that exceeds the time limit of 5 minutes for reasoning about this simple formula. This example underscores the effectiveness of our approach in exploring the rewrite space, thereby uncovering potential inefficiencies or bugs in SMT rewrite systems.

3.5 Bug Detection and Reduction

Types of Bugs. Our methodology enables Aries to detect various types of bugs as follows:

Correctness Bug: This kind of bug will affect the correctness of the solver. For instance, a correctness bug is flagged when the solver under test yields divergent satisfiability results for the mutated formula φ_m and the transformed formula φ_r. Specifically, if φ_r is deemed satisfiable while φ_t is not, or vice versa, this signifies the presence of a correctness bug (*a.k.a, soundness bug*).

Performance Bug: Identified by significant time discrepancies in solving φ_r versus φ_m. Such a gap hints at optimization opportunities, either via new rewrite rules or by improving existing ones. In practice, we can define "abnormal" execution times as those exceeding preset timeframe or ratio thresholds between φ_m and φ_r.

Completeness Bug: In instances where the solver is unexpectedly unable to solve φ_m or φ_r, returning an `unknown` result for φ_r or φ_m but successfully returning `sat` or `unsat` for the other one, a completeness bug is detected.

Crash Bug: A crash bug is noted if the solver terminates unexpectedly while processing φ_m or φ_r, e.g., suffering a segmentation fault or assertion failure.

When a bug is discovered, Aries meticulously documents the bug-triggering formula, along with the altered terms and their respective substitutions employed in the deductive rewriting processes, to aid in subsequent bug analysis practice.

Test Formula Reduction. The bug reduction strategy of Aries is also based on equality saturation. Generally, it decomposes the bug-triggering formula into two distinct parts: the rewritten portion and the unchanged portion, with the intention of minimizing each part individually. Algorithm 4 presents the pseudocode of the bug reduction process. Initially, Aries assigns the bug-triggering formula φ_{bug} to φ_{min} and regard the original formula φ_{orig} as φ_{pass} (Lines 2-3). Next, it constructs an e-graph for each term in $Terms_r$, which is rewritten during the deductive rewriting phase, using the set of rewrite rules *Rules* (Line 5). After that, Aries initializes a list, *InspectList*, to store and manage the terms pending examination, and this list is then populated with the term under current examination (Lines 6-7).

Algorithm 4: Pseudocode for *bug reduction*

```
1  Procedure Reduce(φ_bug, φ_orig, Terms_r, Rules, Solver)
2      φ_min ← φ_bug
3      φ_pass ← φ_orig
4      foreach term_r ∈ Terms_r do
5          EGraph ← BuildEGraph (term_r, Rules)
6          InspectList ← ∅
7          InspectList.add(term_r)
8          while InspectList ≠ ∅ do
9              term ← InspectList.pop(0)
10             term_simplest ← SearchOptimal (EGraph, term, CostFunction)
11             φ_temp ← φ_min.replace(term, term_simplest)
12             if Check (φ_temp, Solver) = bug then
13                 φ_min ← φ_temp
14             else
15                 φ_pass ← SelectComplex (φ_temp, φ_pass)
16                 InspectList.add(term.subterms)
       // Minimize unchanged parts using existing minimization techniques
17     φ_min, φ_pass ← Minimize (φ_min, φ_pass)
18     return φ_min, φ_pass
```

Subsequently, Aries iterates through the terms in *InspectList* from the first to the last, selecting the simplest equivalent term from the e-graph for each term and replacing it in φ_{min} (Lines 10-11). This will results in a simplified formula, φ_{temp}, and if φ_{temp} still triggers the bug, Aries reassigns φ_{min} to φ_{temp}. Otherwise, Aries updates φ_{pass}, and the subterms of the current term are added to *InspectList* for further inspection (Lines 12-16). In particular, to ensure that φ_{pass} remains structurally close to φ_{min}, the `SelectComplex` function selects the more complex formula, measured by the number of S-expressions, between φ_{pass} and φ_{temp} when updating φ_{pass}. Following this iterative process, Aries excludes the irrelevant rewrites and isolates the most suspect rewrites directly implicated in the bug. Subsequently, it applies established minimization techniques to the formula's unchanged portion, ensuring preservation of the segments associated with the isolated suspect rewrites (Line 17). This is to say, only the irrelevant section undergoes minimization. Notably, during this process, φ_{pass} is considered as the reference for checking bugs. The culmination of this procedure is a reduced version of the bug-triggering formula φ_{min}, pinpointing the rewrites most likely contributing to the bug. Ultimately, we can report the reduced bug-triggering formula with the preliminary analysis results to the developers of the solvers. It is important to note that our bug reduction strategy specifically targets formulas generated through deductive rewriting. In other words, bugs or crashes revealed by mimetic mutations fall outside the purview of this process, as they may not directly implicate the rewrite rules. In such cases, we will directly utilize the existing bug reduction techniques to minimize the bug-triggering formulas.

4 Implementation

This section elucidates the pivotal details of Aries's implementation, encompassing the collection of rewrite rules, the employed equality saturation engine, the basic tool for bug reduction, and the configuration of parameters.

Collected Rewrite Rules. This study concentrates on rewrite rules pertaining to theories of integer, real arithmetic, string, and bit-vector theory, while also maintaining the capability to extend these efforts to additional theories. As described in Section 3, we incorporate rewrite rules provided by prior work [Lachnitt et al. 2024] and those inferred by Ruler [Nandi et al. 2021] as complementary sources. These sources contribute a total of 248 and 150 rewrite rules, respectively. Following duplicate removal and verification for SMT-LIB compliance, the final collection of rewrite rules comprises 306 unique entries.

Equality Saturation Engine. Our approach utilizes an equality saturation engine to facilitate the exploration of rewrite space and the derivation of equivalent formulas. In recent years, substantial research has focused on enhancing the efficiency of e-graph construction and the extraction of optimal terms [Koehler et al. 2024; Pal et al. 2023; Willsey et al. 2021; Zhang et al. 2023a, 2022]. In our implementation, we employ *egg* [Willsey et al. 2021], a flexible and high-performance library for equality saturation developed in Rust.

Bug Reduction Tool. To minimize the irrelevant part of the bug-triggering formula, various test case reduction tools can be employed. Here, we adopt C-Reduce [Regehr et al. 2012], an open-source tool known for its effectiveness in minimizing C/C++ programs. Although originally designed for these languages, C-Reduce is versatile and can also be applied to SMT-LIB formulas.

Parameter Configuration. Aries affords users the flexibility to tailor the testing process via several adjustable parameters, which include the iteration count for mimetic mutation ($iteration_m$ in Algorithm 2) and the number of generated test formulas by deductive rewriting. In our implementation, we configured the iteration count $iteration_m$ during mimetic mutation to three, and the number of generated test formulas by deductive rewriting to 10. This is intended to prevent the generation of excessively large test formulas. To identify performance issues, we set a threshold for the solving time ratio, which is conservatively set at 100 in our implementation, informed by prior research [Jiang et al. 2023].

5 Experimental Evaluation

This section presents a comprehensive evaluation of the effectiveness of Aries.

5.1 Evaluation Setup

Research Questions. The experiments conducted aim to answer the following research questions:

- **RQ1:** Can Aries detect authentic bugs in SMT solvers? (Section 5.2)
- **RQ2:** Can Aries effectively explore rewrite space in SMT solvers? (Section 5.3)
- **RQ3:** What is the contribution of key components in Aries? (Section 5.4)
- **RQ4:** How effective is Aries's bug reduction strategy? (Section 5.5)

Test Seeds. In the experiments, we employed formulas from the SMT-LIB official benchmarks[3], which also serve as the basis for the annual SMT Competition [SMT-COMP 2023]. These benchmarks constitute the most comprehensive repository of formal SMT instances available in the SMT-LIB format, providing a standard challenge set for evaluating SMT solvers.

Targeted Solvers. This research utilizes Z3 and cvc5 as the SMT solvers for our experimental evaluation. Notably, these solvers are among the most extensively utilized within the community. Their selection is further justified by their frequent inclusion as subjects in a multitude of previous research endeavors [Park et al. 2021; Sun et al. 2023; Winterer and Su 2024; Winterer et al. 2020a,b].

[3] https://smtlib.cs.uiowa.edu/benchmarks.shtml

Table 2. Status of bugs found in the targeted solvers.

Status	Z3	cvc5	Total
Reported	15	12	27
Confirmed	10	12	22
Fixed	8	8	16
Duplicate	1	0	1

Table 3. Bug types among the reported bugs.

Type	Z3	cvc5	Total
Correctness	6	2	8
Completeness	3	2	5
Performance	1	1	2
Crash	5	7	12

Additionally, owing to the diligent testing and widespread adoption by both developers and end-users, Z3 and cvc5 have demonstrated remarkable stability and soundness [Winterer and Su 2024]. Identifying and reporting bugs in these well-established solvers pose significant challenges, underscoring the importance of our work.

Environment. We conduct our experiments on a machine equipped with an Intel Xeon CPU Gold-5117 processor (48 cores and 128GB RAM), running a Docker container with the Ubuntu 20.04 64-bit operating system.

5.2 RQ1: Bug Detection

The objective of *RQ1* is to explore whether Aries can detect authentic bugs in SMT solvers, which serves as an indicator of the effectiveness of Aries. In this RQ, we use Z3 and cvc5 as the targeted SMT solvers for our experimental evaluation. To unearth previously unknown bugs in these solvers, we employ Aries to validate the latest trunk versions of these solvers, thereby reducing the occurrence of redundant bug reports.

Bug Reporting. We ran Aries for one week to detect bugs in the latest trunk versions of Z3 and cvc5, inspecting suspected issues before reporting them to the developers. Specifically, as described in Section 3.5, Aries identifies and reports the following bug types: correctness bugs, performance bugs, completeness bugs, and crashes. To reduce redundant reports, we carefully review each potential bug identified by Aries prior to submission. Concretely, when crashes occur, we utilize a bug grouping approach to identify those crashes pointing to the same line of code as instances of the same bug and subsequently search for exception information thrown by the solvers in the bug tracking systems for de-duplication. For other types of bugs, similar to prior research [Winterer et al. 2020b], we report one minimized instance at a time according to theory categories and bug types. For example, if we have already reported a correctness issue related to integer arithmetic, we refrain from submitting similar reports until the original issue is resolved. After the reported bug is fixed, we then re-evaluate whether the remaining formulas still trigger issues and decide on further reporting as necessary. Note that, as we employ a conservative bug-reporting strategy as illustrated above, some issues identified by Aries do not yet meet the criteria for reporting; however, these may uncover new bugs in the future.

Statistics. Table 2 presents the status of the bugs discovered by Aries in Z3 and cvc5. In total, Aries has reported 27 bugs across the two solvers, of which 22 bugs have been confirmed or fixed by the developers as of the time of writing. The remaining bugs are pending investigation by the developers. Given our best efforts to de-duplicate the bugs before reporting them, only one bug has been identified as a duplicate issue by the developers. Table 3 illustrates the distribution of bug types among the reported bugs. Additionally, the type of bugs reported by Aries comprises eight correctness bugs, five completeness bugs, two performance bugs, and 12 crashes. The results

Table 4. Distribution of fixed bugs across solver components.

Component	Z3	cvc5	Total
Rewriters	4	5	9
Others	1	3	4
Unknown	3	0	3
All	**8**	**8**	**16**

```
;sigma
(declare-fun x () Int)
(assert (is_int x))
(check-sat)
```

(a)

```
;Rewrite trace of sigma
Rewriter::rewriteTo(THEORY_ARITH,(is_int x))
TheoryArithRewriter::rewriting: ARITH,(is_int x)
setting rewrite of (is_int x) to true
```

(b)

Fig. 7. (a) formula σ and (b) cvc5 rewrite trace of σ.

demonstrate that Aries is capable of exposing authentic bugs in SMT solvers across a variety of bug types. We further manually analyzed the commits associated with resolved issues to determine whether the bugs were indeed related to rewrite systems. Table 4 indicates the distribution of fixed bugs across solver components. Concretely, among the 16 fixed bugs, 13 were explicitly linked to bug-fixing commits by the developers, whereas three were not. Of these, nine were associated with rewrite-related issues according to the commit messages and code changes. The remaining fixed bugs seem to be unrelated to the rewriters. Overall, these results indicate that Aries is effective in uncovering rewrite-related issues in SMT solvers, while also demonstrating its ability to detect bugs in other solver components.

Feedback. Multiple issues reported by Aries have sparked active discussions among developers and got positive feedback. Additionally, the bugs we report are supposed to have a practical impact on the client applications. For instance, as commented by the developers[4], a correctness bug reported by Aries in Z3 could potentially compromise the accuracy of Dafny [Leino 2010], a verification-aware programming language developed by Microsoft Research. In addition, there are also some other users and researchers reflecting the negative impact of our reported bugs on the solvers, which further demonstrates the practical significance of our work.

5.3 RQ2: Exploration of SMT Rewriters

The objective of *RQ2* is to investigate whether Aries can effectively exercise rewrite functionalities in SMT solvers. To achieve this goal, we analyze the rewrite traces triggered by the mutants generated by Aries. For this experiment, we select cvc5 as the subject solver because it provides the `-trace=rewriter` flag, which facilitates the collection of rewrite traces. In contrast, Z3 lacks a similar feature, making it unsuitable for this analysis. Before discussing the experimental setup in detail, we first introduce the rewrite trace format of cvc5 using an example. Consider a formula σ depicted in Figure 7a. The corresponding rewrite traces for this formula, as generated by cvc5, are shown in Figure 7b. These three lines of traces indicate a rewrite process, specifically transforming

[4]https://github.com/dafny-lang/dafny/issues/4053

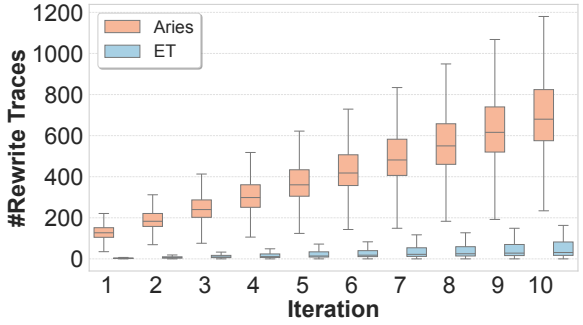


Fig. 8. Unique rewrite traces triggered by the mutants generated by Aries and ET.

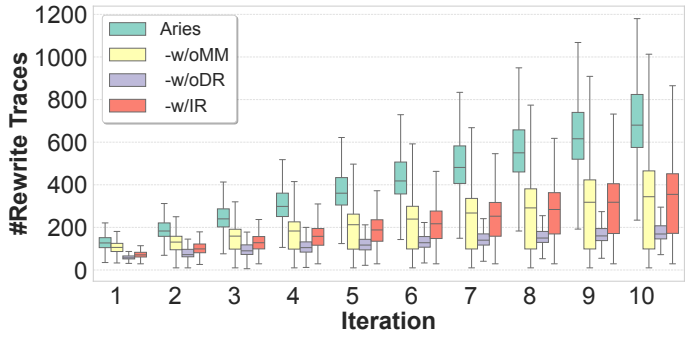


Fig. 9. Unique rewrite traces triggered by the mutants generated by Aries and its three variants.

`(is_int x)` into **`true`**. In this experiment, our primary goal is to evaluate the diversity of rewrites explored by the formulas generated by Aries. Thus, we randomly sample 1,000 seed formulas from the SMT-LIB benchmarks and apply Aries to generate mutants for each formula. We then track the distinct rewrite traces triggered by these mutants over the course of 10 deductive rewriting iterations. To better understand the exploration of rewrite systems, we compare the rewrite traces of the formulas generated by Aries with those generated by the most recently proposed SMT solver validation tool, ET [Winterer and Su 2024]. ET has been shown to effectively uncover bugs in SMT solvers. Because ET relies on a grammar-based strategy that enumerates formulas from simple to increasingly complex structures, we introduce a warm-up phase before making the comparison. In particular, we begin collecting rewrite traces only after the first one million iterations. This threshold follows the evaluation setup in the prior work [Winterer and Su 2024] and corresponds to the most complex formulas used in its experiments.

Figure 8 presents the cumulative number of unique rewrite traces triggered by formulas generated by Aries and ET. The results exhibit a clear trend: as the number of iteration steps increases, the number of unique rewrite traces triggered by the mutants also rises. Specifically, after 10 iterations, mutants generated by Aries can trigger over 1,000 unique rewrite traces, while the increase achieved by ET remains modest. These results highlight the effectiveness of Aries in exploring the rewrite space and exercising the rewrite systems.

5.4 RQ3: Ablation Study

To understand the contribution of each core component in Aries, we conduct an ablation study focusing on three key elements: *mimetic mutation*, *deductive rewriting*, and *expanded rewrite rules*. To this end, we design the following three tool variants, each disabling one specific component:

- Aries$_{\text{w/oMM}}$: This variant disables the mimetic mutation phase, applying only deductive rewriting to the seed formulas.
- Aries$_{\text{w/oDR}}$: This variant disables the deductive rewriting phase, applying only mimetic mutation to the seed formulas.
- Aries$_{\text{w/IR}}$: This variant uses only the rewrite rules inferred by Ruler [Nandi et al. 2021], without incorporating the manually curated rules in Aries.

We evaluate these variants using the same experimental setup described in *RQ2*. Specifically, we compare the sets of mutants produced by each variant in terms of the unique rewrite traces they trigger during solver execution. This analysis helps quantify how each component contributes to the diversity and effectiveness of the generated mutations. The results of this comparison are summarized in Figure 9.

From the figure, we observe that Aries generates a higher number of unique rewrite traces compared to the three variants, across all mutation steps. Among the variants, Aries$_{\text{w/oMM}}$ produces a number of traces more comparable to Aries, while Aries$_{\text{w/oDR}}$ shows a notably lower count. This suggests that deductive rewriting plays a more essential role in triggering diverse rewrite behaviors. However, mimetic mutation also demonstrates an important role. Without it, some seed formulas fail to trigger any rewrite activity during the iterations, leaving the lower whisker end at zero. Together, these results suggest that while deductive rewriting is key to enabling rewrite exploration, combining it with mimetic mutation leads to the most effective exploration. We also observe a drop in rewrite traces when using Aries$_{\text{w/IR}}$ compared to the full version of Aries. This highlights the importance of maintaining a comprehensive set of rewrite rules. Specifically, while the inferred rules alone can still trigger rewrite behaviors, demonstrating a degree of generality, they are clearly insufficient to capture the full capabilities of the complete rule set.

5.5 RQ4: Bug Reduction

This RQ aims to investigate the effectiveness of Aries's bug reduction strategy. Based on the prior work [Mansur et al. 2020], we design this experiment to evaluate the bug-triggering formula reduction capability on the bug-triggering formulas generated by Aries. Concretely, we inspect the bug-triggering formulas of the 22 confirmed issues discovered by Aries in the previous experiment. Notably, the 12 crashes are excluded since they are reduced using existing tools as illustrated in Section 3.5. Subsequently, we apply Aries to reduce these formulas and compare the size and **`assert`** statement count of the reduced formulas with the original ones. We also compare our tool Aries with the state-of-the-art SMT-LIB language reduction tool, ddSMT [Kremer et al. 2021], in this experiment. Here, we employ its latest version, ddSMT-2.0.5.

Table 5 showcases the results. We observe that Aries is capable of reducing the size of the bug-triggering formulas and the number of **`assert`** statements effectively. Aries decreased formula size to 253.0 bytes and **`assert`** statements to 1.1, marking a significant reduction from the original formulas. Although ddSMT achieved a better reduction in case size, it does not have Aries's ability to directly identify the specific rewrite rules related to the bug, which is a distinct advantage of our approach. For example, Figure 10 illustrates two cases generated by Aries's reduction strategy. Particularly, the first case cannot be solved by Z3, as shown in Figure 10a, and returns `unknown` (Figure 10c). The reduction strategy of Aries suggests that the issue could be addressed by applying the rewrite $(str.\ <=\ ""\ s) \rightsquigarrow true$, since an empty string cannot lexicographically exceed other

Table 5. Size of original and minimized bug-triggering instances. Instance size is shown in terms of the number of bytes and number of assertions.

	Original		Aries		ddSMT	
	Bytes	Asserts	Bytes	Asserts	Bytes	Asserts
Average	1784.9	2.8	253.0	1.1	168.1	1.2

```
(declare-fun v () String)
(assert (str.<= "" (str.replace_all v "i" v)))
(check-sat)
```

(a) Bug-triggering case reduced by Aries.

```

(assert true)
(check-sat)
```

(b) Passing test case indicated by Aries.

```
$ z3 reduced.smt2
unknown

$ z3 pass.smt2
sat
```

(c) Command lines.

```
br_status seq_rewriter::mk_str_lt(...) {
...
    if (str().is_empty(a)) {
        result = m().mk_not(m().mk_eq(a, b));
        return BR_DONE;
     }
```

(d) Patched rewrite rule in Z3.

Fig. 10. Reduced bug sample triggering a completeness issue identified by Aries. #7446

strings. As a result, we reported the issue and mentioned the potential solution to the Z3 developers. Subsequently, a developer implemented the suggested fix and submitted a commit, as seen in Figure 10d. This demonstrates the practical value of Aries's reduction strategy.

6 Discussion

6.1 Automated Rule Collection

SMT solvers are under active development, and their rewrite systems evolve correspondingly. To ensure that Aries remains effective as these solvers advance, it is essential to maintain an up-to-date set of rewrite rules. While manual efforts and community contributions remain valuable, we also explore automated methods to reduce the maintenance burden. In particular, we investigate whether large language models (LLMs) can help identify and summarize changes to solver rewrite rules. Recent progress in LLMs shows that they can understand both natural language and source code, making them a promising tool for analyzing code repositories and extracting meaningful updates with minimal manual effort.

As a preliminary study, we examined the recent development history of cvc5, focusing on pull requests (PRs) submitted between April and June 2025 that introduced new rewrite rules. We identified relevant PRs by searching for the keyword "add rewrite," yielding 11 potential candidates. After manual inspection, we confirmed that two of these PRs, PR#11794 and PR#11782, explicitly added rewrite rules, introducing four new rules in total. The remaining PRs involved other types of modifications to the rewriter. We then used GPT-4o [Hurst et al. 2024], one of the most capable

```
### You are a code analysis assistant with expertise in SMT
solver implementations.
Please analyze the following merged Pull Request in [SOLVER].
```

```
# PR Description:
[DESCRIPTION]
```

```
# Code Diff:
[DIFF]
```

```
### Your task is to identify whether any rewrite rules have
been added or modified within rewriter components. If so,
extract and summarize each rewrite rule in the following format:
```

```
# Format:
[EXAMPLE]
```

Fig. 11. The prompt used in rule analysis.

LLMs, to analyze these PRs, guided by the prompt shown in Figure 11. Ultimately, we observed that it successfully extracted all four newly added rules, accurately capturing their structure and semantics. These results indicate that LLMs can serve as promising assistants for tracking solver rewrite rules, supporting the long-term maintainability of Aries.

6.2 Effect of *Mimetic Mutation* on Satisfiability

As mimetic mutation does not preserve the satisfiability of generated mutants, we are also curious about its influence on formula satisfiability. To this end, we examine the satisfiability of formulas sampled from the SMT-LIB benchmarks both before and after applying mimetic mutation. Our observations indicate that, in practice, mimetic mutation tends to produce unsatisfiable mutants from satisfiable seeds, but it does not significantly affect the overall ratio of satisfiable to unsatisfiable formulas. Specifically, our analysis shows that fewer than 2% of initially unsatisfiable formulas became satisfiable after mimetic mutation, while approximately 5% of originally satisfiable formulas became unsatisfiable. We deduce that this minimal shift in satisfiability to the limited number of terms modified by mimetic mutation, as Aries mutates only three terms under our default setting. To further assess this hypothesis, we conducted additional experiments with a higher mutation intensity, increasing the number of terms mutated in the seed formulas. For instance, with ten terms mutated, about 20% of the satisfiable seeds were transformed into unsatisfiable ones, illustrating a more evident effect on the satisfiability ratio.

6.3 Threats to Validity

Our investigation faces several threats to validity that merit attention. First, the efficacy of Aries is contingent upon the extensive collection of rewrite rules. To validate the SMT rewrite systems comprehensively, we amassed a significant corpus of rewrite rules from both existing collections and the application of rule inference techniques. Second, the correctness of these rules is paramount; any inaccuracies may lead Aries to elicit false positives. We addressed this risk by rigorously verifying the rules' correctness through their encoding into SMT instances, subsequently rectifying or discarding any that were found to be flawed. Third, the parameter settings in Aries may affect its performance. In particular, we configured these parameters in this study to prevent the generation of excessively large test formulas, which could degrade testing throughput or pose challenges for

bug minimization. While our evaluation confirms that Aries is effective with the chosen settings, alternative configurations may uncover additional issues.

6.4 Ethical Consideration

To assist in the developers' debugging efforts, we have devised a bug-triggering formula reduction strategy designed to reduce the complexity of bug-triggering formulas, simultaneously emphasizing their critical elements. Furthermore, as detailed in Section 5.2, we meticulously eliminate duplicate bugs by adhering to strict guidelines, expecting that redundant issues are not submitted to developers. As a result, only one issue among the more than 20 reports has been classified as a duplicate by the developers.

6.5 Limitation and Future Work

An important limitation of Aries lies in the inherent incompleteness. While it is theoretically conceivable to achieve bounded completeness for certain decidable theories, assuming access to the solver's full rewrite specification and exhaustive formula exploration, this remains practically infeasible. Achieving such coverage would require a complete, formalized set of rewrite rules for a theory and exhaustive derivation of all formulas from a rich seed set (e.g., the entire SMT-LIB benchmark suite). In practice, however, solvers evolve, and exhaustive enumeration of formulas is computationally prohibitive. As such, we prioritize discovering real and impactful bugs over guaranteeing completeness. We believe this trade-off offers a more scalable and pragmatic path to enhancing solver reliability.

This work can also be further explored in the future. Specifically, while our validation campaign has primarily focused on Z3 and cvc5, we emphasize that Aries is general and can be applied to other SMT solvers. For instance, we have conducted a tentative validation on OpenSMT2 [Hyvärinen et al. 2016], where Aries successfully uncovered two bugs. A broader application of Aries to additional solvers is planned for future work. Our current implementation targets rewrite rules in arithmetic, strings, and bit-vectors. However, in principle, Aries is designed to support other SMT-LIB theories as well. Extending our rule database and seed generation to cover a wider set of theories is a natural next step, enabling a more comprehensive validation of solver behavior across different domains. Furthermore, there is room to improve the efficiency and effectiveness of our validation and delta-debugging algorithms. Future work will explore new heuristics, optimizations, and learning-based techniques to reduce redundancy and enhance bug isolation. Additionally, we envision extending Aries beyond SMT solvers. Its framework is adaptable to other rewrite systems, such as compilers and database management systems. Investigating these broader applications represents a promising avenue for future research.

7 Related Work

SMT Solver Testing. The widespread adoption of SMT solvers in various domains [Cadar et al. 2008; Sigurbjarnarson et al. 2016; Sun et al. 2024] underscores the critical importance of their correctness and performance. Consequently, various techniques have been developed to ensure the quality of SMT solvers. One of the earliest techniques for SMT solver fuzzing is FuzzSMT, proposed by Brummayer and Biere [Brummayer and Biere 2009]. This grammar-based blackbox fuzzing tool has been followed by recent efforts, such as Murxla [Niemetz et al. 2022]. Additionally, mutation-based fuzzing techniques have also shown promise in detecting SMT solver bugs [Mansur et al. 2020; Winterer et al. 2020a,b]. However, these techniques cannot systematically test the SMT solvers. Therefore, the latest research, ET [Winterer and Su 2024], proposes an advanced validation approach for SMT solvers by generating formulas via grammar-based enumeration to validate the

solvers. Different from these existing techniques, Aries specifically focuses on validating the SMT rewrite systems via *Rewrite Space Exploration*. In particular, we devise a novel approach including *mimetic mutation* and *deductive rewriting* to generate effective formulas to exercise the rewriters.

Rewrite Rules in SMT Solvers. Rewriters in SMT solvers are essential components that preprocess formulas for efficient solving. Therefore, to ensure the correctness and efficiency of rewriters, researchers have proposed various techniques to automatically synthesize rules for rewriters. For instance, Ruler [Nandi et al. 2021] is a tool that can synthesize rewrite rules from scratch. Specifically, Ruler uses e-graphs to compactly encode large sets of terms and equality saturation to efficiently filter and minimize rulesets. In contrast, Aries focuses on validating the rewriters in SMT solvers by generating effective test formulas to explore rewrite space.

Metamorphic Testing. Metamorphic Testing (MT) is a widely adopted software testing technique designed to address the test oracle problem [Chen et al. 2020]. It has been successfully applied across various domains, such as compilers [Li et al. 2023; Yang et al. 2019], debuggers [Yang et al. 2025], quantum computing platforms [Paltenghi and Pradel 2023], and database management systems [Jiang and Su 2024]. In our work, we leverage the principles of metamorphic testing to generate semantically equivalent formulas using rewrite rules. Particularly, the formulas produced through *deductive rewriting* are designed to preserve the satisfiability results of the original formulas. A notable advantage of our approach lies in its capacity to provide robust oracles given the semantical equivalence, enabling the effective detection of bugs in SMT rewrite systems. These include issues related to correctness, completeness, and performance, which are typically insufficiently supported by existing techniques.

8 Conclusion

This paper has introduced Aries, an innovative approach for validating SMT solver rewrite systems by leveraging the concepts of *mimetic mutation* and *deductive rewriting*. By employing the two strategies, we can generate effective formulas to explore the rewrite space of the SMT solver. Additionally, we have developed a customized bug-triggering formula reduction strategy to facilitate the debugging process. Implemented as a practical testing tool, Aries has undergone comprehensive evaluations with state-of-the-art SMT solvers. By conducting a validation campaign, Aries has uncovered 27 new bugs in the SMT solvers, with 22 of these bugs being either confirmed or fixed by the developers. The majority of these issues are attributed to the rewrite systems. Moreover, our findings reveal that Aries effectively exercises the rewrite systems within SMT solvers.

Data-Availability Statement

The tool Aries with the usage is publicly available [Sun et al. 2025].

Acknowledgments

We thank the anonymous reviewers for their valuable and constructive feedback. We also greatly appreciate the Z3 and cvc5 development teams, in particular Nikolaj Bjørner, Andrew Reynolds, Clark Barrett, and Martin Brain, for their support and for carefully examining the bugs we reported. Yibiao Yang is the corresponding author. This work was supported in part by the National Natural Science Foundation of China (Grants 624B2067, 62572226, 62072194, 62172205), the Jiangsu Natural Science Foundation (Grant BK20231402), the Collaborative Innovation Center of Novel Software Technology and Industrialization, and the Fundamental Research Funds for the Central Universities (XJ2024003301, XJ2025000601).

References

2024. cvc5. https://github.com/cvc5/cvc5.

Supun Abeysinghe, Anxhelo Xhebraj, and Tiark Rompf. 2024. Flan: An Expressive and Efficient Datalog Compiler for Program Analysis. *Proc. ACM Program. Lang.* 8, POPL (2024), 2577–2609. https://doi.org/10.1145/3632928

Haniel Barbosa, Clark W. Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. 2022. cvc5: A Versatile and Industrial-Strength SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems - 28th International Conference, TACAS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 13243)*, Dana Fisman and Grigore Rosu (Eds.). Springer, 415–442. https://doi.org/10.1007/978-3-030-99524-9_24

Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2022. *The satisfiability modulo theories library (SMT-LIB).* Retrieved 2022-08-18 from http://smtlib.cs.uiowa.edu/

Clark Barrett and Cesare Tinelli. 2018. Satisfiability modulo theories. In *Handbook of model checking*. Springer, 305–343.

Mohamed Amine Batoun, Ka Lai Yung, Yuan Tian, and Mohammed Sayagh. 2023. An Empirical Study on GitHub Pull Requests' Reactions. *ACM Trans. Softw. Eng. Methodol.* 32, 6 (2023), 146:1–146:35. https://doi.org/10.1145/3597208

Martin Brain. 2024. How to know the implemented rewrite rules in cvc5? https://github.com/cvc5/cvc5/discussions/10275#discussioncomment-8159588.

Robert Brummayer and Armin Biere. 2009. Fuzzing and Delta-Debugging SMT Solvers. In *Proceedings of the 7th International Workshop on Satisfiability Modulo Theories* (Montreal, Canada) *(SMT '09)*. Association for Computing Machinery, New York, NY, USA, 1–5. https://doi.org/10.1145/1670412.1670413

Cristian Cadar, Daniel Dunbar, and Dawson R. Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *8th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2008, December 8-10, 2008, San Diego, California, USA, Proceedings*, Richard Draves and Robbert van Renesse (Eds.). USENIX Association, 209–224. http://www.usenix.org/events/osdi08/tech/full_papers/cadar/cadar.pdf

David Cao, Rose Kunkel, Chandrakana Nandi, Max Willsey, Zachary Tatlock, and Nadia Polikarpova. 2023. babble: Learning Better Abstractions with E-Graphs and Anti-unification. *Proc. ACM Program. Lang.* 7, POPL (2023), 396–424. https://doi.org/10.1145/3571207

Renze Chen, Zijian Ding, Size Zheng, Chengrui Zhang, Jingwen Leng, Xuanzhe Liu, and Yun Liang. 2024. MAGIS: Memory Optimization via Coordinated Graph Transformation and Scheduling for DNN. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 607–621. https://doi.org/10.1145/3620666.3651330

Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2018, Carlsbad, CA, USA, October 8-10, 2018*, Andrea C. Arpaci-Dusseau and Geoff Voelker (Eds.). USENIX Association, 578–594. https://www.usenix.org/conference/osdi18/presentation/chen

Tsong Yueh Chen, S. C. Cheung, and Siu-Ming Yiu. 2020. Metamorphic Testing: A New Approach for Generating Next Test Cases. *CoRR* abs/2002.12543 (2020). arXiv:2002.12543 https://arxiv.org/abs/2002.12543

Jianyi Cheng, Samuel Coward, Lorenzo Chelini, Rafael Barbalho, and Theo Drane. 2024. SEER: Super-Optimization Explorer for High-Level Synthesis using E-graph Rewriting. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 1029–1044. https://doi.org/10.1145/3620665.3640392

Leonardo Mendonça de Moura and Nikolaj S. Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29-April 6, 2008. Proceedings (Lecture Notes in Computer Science, Vol. 4963)*, C. R. Ramakrishnan and Jakob Rehof (Eds.). Springer, 337–340. https://doi.org/10.1007/978-3-540-78800-3_24

James C. Hoe and Arvind. 1999. Hardware Synthesis from Term Rewriting Systems. In *VLSI: Systems on a Chip, IFIP TC10/WG10.5 Tenth International Conference on Very Large Scale Integration (VLSI '99), December 1-4, 1999, Lisbon, Portugal (IFIP Conference Proceedings, Vol. 162)*, L. Miguel Silveira, Srinivas Devadas, and Ricardo Augusto da Luz Reis (Eds.). Kluwer, 595–619.

Aaron Hurst, Adam Lerer, Adam P. Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, Aleksander Madry, Alex Baker-Whitcomb, Alex Beutel, Alex Borzunov, Alex Carney, Alex Chow, Alex Kirillov, Alex Nichol, Alex Paino, Alex Renzin, Alex Tachard Passos, Alexander Kirillov, Alexi Christakis,

Alexis Conneau, Ali Kamali, Allan Jabri, Allison Moyer, Allison Tam, Amadou Crookes, Amin Tootoonchian, Ananya Kumar, Andrea Vallone, Andrej Karpathy, Andrew Braunstein, Andrew Cann, Andrew Codispoti, Andrew Galu, Andrew Kondrich, Andrew Tulloch, Andrey Mishchenko, Angela Baek, Angela Jiang, Antoine Pelisse, Antonia Woodford, Anuj Gosalia, Arka Dhar, Ashley Pantuliano, Avi Nayak, Avital Oliver, Barret Zoph, Behrooz Ghorbani, Ben Leimberger, Ben Rossen, Ben Sokolowsky, Ben Wang, Benjamin Zweig, Beth Hoover, Blake Samic, Bob McGrew, Bobby Spero, Bogo Giertler, Bowen Cheng, Brad Lightcap, Brandon Walkin, Brendan Quinn, Brian Guarraci, Brian Hsu, Bright Kellogg, Brydon Eastman, Camillo Lugaresi, Carroll L. Wainwright, Cary Bassin, Cary Hudson, Casey Chu, Chad Nelson, Chak Li, Chan Jun Shern, Channing Conger, Charlotte Barette, Chelsea Voss, Chen Ding, Cheng Lu, Chong Zhang, Chris Beaumont, Chris Hallacy, Chris Koch, Christian Gibson, Christina Kim, Christine Choi, Christine McLeavey, Christopher Hesse, Claudia Fischer, Clemens Winter, Coley Czarnecki, Colin Jarvis, Colin Wei, Constantin Koumouzelis, and Dane Sherburn. 2024. GPT-4o System Card. *CoRR* abs/2410.21276 (2024). https://doi.org/10.48550/ARXIV.2410.21276 arXiv:2410.21276

Antti E. J. Hyvärinen, Matteo Marescotti, Leonardo Alt, and Natasha Sharygina. 2016. OpenSMT2: An SMT Solver for Multi-core and Cloud Computing. In *Theory and Applications of Satisfiability Testing - SAT 2016 - 19th International Conference, Bordeaux, France, July 5-8, 2016, Proceedings (Lecture Notes in Computer Science, Vol. 9710)*, Nadia Creignou and Daniel Le Berre (Eds.). Springer, 547–553. https://doi.org/10.1007/978-3-319-40970-2_35

Shuyao Jiang, Ruiying Zeng, Zihao Rao, Jiazhen Gu, Yangfan Zhou, and Michael R. Lyu. 2023. Revealing Performance Issues in Server-Side WebAssembly Runtimes Via Differential Testing. In *38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023*. IEEE, 661–672. https://doi.org/10.1109/ASE56229.2023.00088

Zu-Ming Jiang and Zhendong Su. 2024. Detecting Logic Bugs in Database Engines via Equivalent Expression Transformation. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, Ada Gavrilovska and Douglas B. Terry (Eds.). USENIX Association, 821–835. https://www.usenix.org/conference/osdi24/presentation/jiang

Thomas Koehler, Andrés Goens, Siddharth Bhat, Tobias Grosser, Phil Trinder, and Michel Steuwer. 2024. Guided Equality Saturation. *Proc. ACM Program. Lang.* 8, POPL (2024), 1727–1758. https://doi.org/10.1145/3632900

Gereon Kremer, Aina Niemetz, and Mathias Preiner. 2021. ddSMT 2.0: Better Delta Debugging for the SMT-LIBv2 Language and Friends. In *Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20-23, 2021, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 12760)*, Alexandra Silva and K. Rustan M. Leino (Eds.). Springer, 231–242. https://doi.org/10.1007/978-3-030-81688-9_11

Hanna Lachnitt, Mathias Fleury, Leni Aniva, Andrew Reynolds, Haniel Barbosa, Andres Nötzli, Clark W. Barrett, and Cesare Tinelli. 2024. IsaRare: Automatic Verification of SMT Rewrites in Isabelle/HOL. In *Tools and Algorithms for the Construction and Analysis of Systems - 30th International Conference, TACAS 2024, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2024, Luxembourg City, Luxembourg, April 6-11, 2024, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 14570)*, Bernd Finkbeiner and Laura Kovács (Eds.). Springer, 311–330. https://doi.org/10.1007/978-3-031-57246-3_17

K. Rustan M. Leino. 2010. Dafny: An Automatic Program Verifier for Functional Correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning - 16th International Conference, LPAR-16, Dakar, Senegal, April 25-May 1, 2010, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 6355)*, Edmund M. Clarke and Andrei Voronkov (Eds.). Springer, 348–370. https://doi.org/10.1007/978-3-642-17511-4_20

Cong Li, Yanyan Jiang, Chang Xu, and Zhendong Su. 2023. Validating JIT Compilers via Compilation Space Exploration. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace (Eds.). ACM, 66–79. https://doi.org/10.1145/3600006.3613140

Muhammad Numair Mansur, Maria Christakis, Valentin Wüstholz, and Fuyuan Zhang. 2020. Detecting critical bugs in SMT solvers using blackbox mutational fuzzing. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 701–712. https://doi.org/10.1145/3368089.3409763

Alexander Nadel. 2014. Bit-Vector Rewriting with Automatic Rule Generation. In *Computer Aided Verification - 26th International Conference, CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 18-22, 2014. Proceedings (Lecture Notes in Computer Science, Vol. 8559)*, Armin Biere and Roderick Bloem (Eds.). Springer, 663–679. https://doi.org/10.1007/978-3-319-08867-9_44

Chandrakana Nandi, Max Willsey, Amy Zhu, Yisu Remy Wang, Brett Saiki, Adam Anderson, Adriana Schulz, Dan Grossman, and Zachary Tatlock. 2021. Rewrite rule inference using equality saturation. *Proc. ACM Program. Lang.* 5, OOPSLA (2021), 1–28. https://doi.org/10.1145/3485496

Greg Nelson and Derek C. Oppen. 1980. Fast Decision Procedures Based on Congruence Closure. *J. ACM* 27, 2 (1980), 356–364. https://doi.org/10.1145/322186.322198

Luke Nelson, Helgi Sigurbjarnarson, Kaiyuan Zhang, Dylan Johnson, James Bornholt, Emina Torlak, and Xi Wang. 2017. Hyperkernel: Push-Button Verification of an OS Kernel. In *Proceedings of the 26th Symposium on Operating Systems*

Principles, Shanghai, China, October 28-31, 2017. ACM, 252–269. https://doi.org/10.1145/3132747.3132748

Aina Niemetz, Mathias Preiner, and Clark W. Barrett. 2022. Murxla: A Modular and Highly Extensible API Fuzzer for SMT Solvers. In *Computer Aided Verification - 34th International Conference, CAV 2022, Haifa, Israel, August 7-10, 2022, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 13372)*, Sharon Shoham and Yakir Vizel (Eds.). Springer, 92–106. https://doi.org/10.1007/978-3-031-13188-2_5

Andres Nötzli, Haniel Barbosa, Aina Niemetz, Mathias Preiner, Andrew Reynolds, Clark W. Barrett, and Cesare Tinelli. 2022. Reconstructing Fine-Grained Proofs of Rewrites Using a Domain-Specific Language. In *22nd Formal Methods in Computer-Aided Design, FMCAD 2022, Trento, Italy, October 17-21, 2022*, Alberto Griggio and Neha Rungta (Eds.). IEEE, 65–74. https://doi.org/10.34727/2022/ISBN.978-3-85448-053-2_12

Andres Nötzli, Andrew Reynolds, Haniel Barbosa, Aina Niemetz, Mathias Preiner, Clark W. Barrett, and Cesare Tinelli. 2019. Syntax-Guided Rewrite Rule Enumeration for SMT Solvers. In *Theory and Applications of Satisfiability Testing - SAT 2019 - 22nd International Conference, SAT 2019, Lisbon, Portugal, July 9-12, 2019, Proceedings (Lecture Notes in Computer Science, Vol. 11628)*, Mikolás Janota and Inês Lynce (Eds.). Springer, 279–297. https://doi.org/10.1007/978-3-030-24258-9_20

Anjali Pal, Brett Saiki, Ryan Tjoa, Cynthia Richey, Amy Zhu, Oliver Flatt, Max Willsey, Zachary Tatlock, and Chandrakana Nandi. 2023. Equality Saturation Theory Exploration à la Carte. *Proc. ACM Program. Lang.* 7, OOPSLA2 (2023), 1034–1062. https://doi.org/10.1145/3622834

Matteo Paltenghi and Michael Pradel. 2023. MorphQ: Metamorphic Testing of the Qiskit Quantum Computing Platform. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2413–2424. https://doi.org/10.1109/ICSE48619.2023.00202

Jiwon Park, Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2021. Generative type-aware mutation for testing SMT solvers. *Proc. ACM Program. Lang.* 5, OOPSLA (2021), 1–19. https://doi.org/10.1145/3485529

John Regehr, Yang Chen, Pascal Cuoq, Eric Eide, Chucky Ellison, and Xuejun Yang. 2012. Test-case reduction for C compiler bugs. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '12, Beijing, China - June 11 - 16, 2012*, Jan Vitek, Haibo Lin, and Frank Tip (Eds.). ACM, 335–346. https://doi.org/10.1145/2254064.2254104

Helgi Sigurbjarnarson, James Bornholt, Emina Torlak, and Xi Wang. 2016. Push-Button Verification of File Systems via Crash Refinement. In *12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016*, Kimberly Keeton and Timothy Roscoe (Eds.). USENIX Association, 1–16. https://www.usenix.org/conference/osdi16/technical-sessions/presentation/sigurbjarnarson

SMT-COMP. 2023. *The International SMT Competition.* Retrieved 2023-12-1 from https://smt-comp.github.io/2023/

Maolin Sun, Yibiao Yang, Ming Wen, Yongcong Wang, Yuming Zhou, and Hai Jin. 2023. Validating SMT Solvers via Skeleton Enumeration Empowered by Historical Bug-Triggering Inputs. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 69–81. https://doi.org/10.1109/ICSE48619.2023.00018

Maolin Sun, Yibiao Yang, Jiangchang Wu, and Yuming Zhou. 2025. *Artifact for OOPSLA'25 paper "Validating SMT Rewriters via Rewrite Space Exploration Supported by Generative Equality Saturation"*. https://doi.org/10.5281/zenodo.15754886

Xudong Sun, Wenjie Ma, Jiawei Tyler Gu, Zicheng Ma, Tej Chajed, Jon Howell, Andrea Lattuada, Oded Padon, Lalith Suresh, Adriana Szekeres, and Tianyin Xu. 2024. Anvil: Verifying Liveness of Cluster Management Controllers. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, Ada Gavrilovska and Douglas B. Terry (Eds.). USENIX Association, 649–666. https://www.usenix.org/conference/osdi24/presentation/sun-xudong

Ross Tate, Michael Stepp, Zachary Tatlock, and Sorin Lerner. 2009. Equality saturation: a new approach to optimization. In *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2009, Savannah, GA, USA, January 21-23, 2009*, Zhong Shao and Benjamin C. Pierce (Eds.). ACM, 264–276. https://doi.org/10.1145/1480881.1480915

Alexa VanHattum, Rachit Nigam, Vincent T. Lee, James Bornholt, and Adrian Sampson. 2021. Vectorization for digital signal processors via equality saturation. In *ASPLOS '21: 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Virtual Event, USA, April 19-23, 2021*, Tim Sherwood, Emery D. Berger, and Christos Kozyrakis (Eds.). ACM, 874–886. https://doi.org/10.1145/3445814.3446707

Yang Wang, Peng Zhang, Maolin Sun, Zeyu Lu, Yibiao Yang, Yutian Tang, Junyan Qian, Zhi Li, and Yuming Zhou. 2023. Uncovering Bugs in Code Coverage Profilers via Control Flow Constraint Solving. *IEEE Trans. Software Eng.* 49, 11 (2023), 4964–4987. https://doi.org/10.1109/TSE.2023.3321381

Zhaoguo Wang, Zhou Zhou, Yicun Yang, Haoran Ding, Gansen Hu, Ding Ding, Chuzhe Tang, Haibo Chen, and Jinyang Li. 2022. WeTune: Automatic Discovery and Verification of Query Rewrite Rules. In *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Zachary G. Ives, Angela Bonifati, and Amr El Abbadi (Eds.). ACM, 94–107. https://doi.org/10.1145/3514221.3526125

Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. egg: Fast and extensible equality saturation. *Proc. ACM Program. Lang.* 5, POPL (2021), 1–29. https://doi.org/10.1145/3434304

Dominik Winterer and Zhendong Su. 2024. Validating SMT Solvers for Correctness and Performance via Grammar-Based Enumeration. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 355 (Oct. 2024), 24 pages. https://doi.org/10.1145/3689795

Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2020a. On the unusual effectiveness of type-aware operator mutations for testing SMT solvers. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 193:1–193:25. https://doi.org/10.1145/3428261

Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2020b. Validating SMT solvers via semantic fusion. In *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2020, London, UK, June 15-20, 2020*, Alastair F. Donaldson and Emina Torlak (Eds.). ACM, 718–730. https://doi.org/10.1145/3385412.3385985

Yibiao Yang, Yanyan Jiang, Zhiqiang Zuo, Yang Wang, Hao Sun, Hongmin Lu, Yuming Zhou, and Baowen Xu. 2019. Automatic Self-Validation for Code Coverage Profilers. In *34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019, San Diego, CA, USA, November 11-15, 2019*. IEEE, 79–90. https://doi.org/10.1109/ASE.2019.00018

Yibiao Yang, Maolin Sun, Jiangchang Wu, Qingyang Li, and Yuming Zhou. 2025. Debugger Toolchain Validation via Cross-Level Debugging. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1, ASPLOS 2025, Rotterdam, The Netherlands, 30 March 2025 - 3 April 2025*, Lieven Eeckhout, Georgios Smaragdakis, Kaitai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach (Eds.). ACM, 280–294. https://doi.org/10.1145/3669940.3707271

Peisen Yao, Jinguo Zhou, Xiao Xiao, Qingkai Shi, Rongxin Wu, and Charles Zhang. 2024. Falcon: A Fused Approach to Path-Sensitive Sparse Data Dependence Analysis. *Proc. ACM Program. Lang.* 8, PLDI (2024), 567–592. https://doi.org/10.1145/3656400

Mengxiao Zhang, Zhenyang Xu, Yongqiang Tian, Yu Jiang, and Chengnian Sun. 2023b. PPR: Pairwise Program Reduction. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3-9, 2023*, Satish Chandra, Kelly Blincoe, and Paolo Tonella (Eds.). ACM, 338–349. https://doi.org/10.1145/3611643.3616275

Yihong Zhang, Yisu Remy Wang, Oliver Flatt, David Cao, Philip Zucker, Eli Rosenthal, Zachary Tatlock, and Max Willsey. 2023a. Better Together: Unifying Datalog and Equality Saturation. *Proc. ACM Program. Lang.* 7, PLDI (2023), 468–492. https://doi.org/10.1145/3591239

Yihong Zhang, Yisu Remy Wang, Max Willsey, and Zachary Tatlock. 2022. Relational e-matching. *Proc. ACM Program. Lang.* 6, POPL (2022), 1–22. https://doi.org/10.1145/3498696

Received 2025-03-26; accepted 2025-08-12